OOPs

Interview Preparation Guide

Top 20 Most Asked Questions & Answers
Intermediate to Advanced Level | With Easy Explanations

Level	Count	Topics Covered
Intermediate	12	Class vs Object, Encapsulation, Abstraction, Inheritance, Polymorphism, Overloading vs Overriding, Constructors, Access Modifiers
Advanced	8	Dynamic Method Dispatch, Diamond Problem, SOLID Principles, Virtual Functions, Object Slicing, Static vs Dynamic Binding, Design Patterns

Structured for quick revision before technical interviews

Table of Contents

Q1.	What is Object-Oriented Programming, and what are its 4 main pillars?	Intermediate
Q2.	What is the difference between a Class and an Object?	Intermediate
Q3.	What is Encapsulation, and why is it useful?	Intermediate
Q4.	What is Abstraction, and how is it different from Encapsulation?	Intermediate
Q5.	What is Inheritance, and what are its types?	Intermediate
Q6.	What is Polymorphism, and what are its types?	Intermediate
Q7.	What is the difference between Method Overloading and Method Overriding?	Intermediate
Q8.	What is a Constructor, and what are its key characteristics?	Intermediate
Q9.	What are Access Modifiers (public, private, protected)?	Intermediate
Q10.	What is the difference between an Abstract Class and an Interface?	Intermediate
Q11.	What is the purpose of the 'this' and 'super' keywords?	Intermediate
Q12.	What is the difference between Association, Aggregation, and Composition?	Intermediate
Q13.	How does Method Overriding achieve runtime polymorphism internally (dynamic method dispatch)?	Advanced
Q14.	What is the Diamond Problem in Multiple Inheritance, and how is it resolved?	Advanced
Q15.	What are the SOLID principles of Object-Oriented Design?	Advanced
Q16.	What is a Virtual Function, and what is a Pure Virtual Function?	Advanced
Q17.	What is Object Slicing?	Advanced
Q18.	What is the difference between Static Binding and Dynamic Binding?	Advanced
Q19.	What is Coupling and Cohesion in OOP design, and why does low coupling with high cohesion matter?	Advanced
Q20.	What is a Design Pattern? Explain the Singleton Pattern as an example.	Advanced

Q1 **OOP Basics** Intermediate

What is Object-Oriented Programming, and what are its 4 main pillars?

ANSWER

Object-Oriented Programming (OOP) is a programming paradigm that organizes code around 'objects' — bundles of data (attributes) and behavior (methods) — instead of just a sequence of functions/instructions, making code more modular, reusable, and easier to maintain.

The 4 main pillars are: Encapsulation (bundling data and methods together, hiding internal details), Abstraction (showing only essential features, hiding complexity), Inheritance (a class acquiring properties of another class), and Polymorphism (the same interface behaving differently based on the object using it).

EASY EXPLANATION: Think of a car: instead of writing separate, disconnected instructions for 'turn wheel', 'press pedal', 'honk', OOP groups everything related to a car into one 'Car' object that has its own properties (color, speed) and behaviors (drive, honk) bundled together — like a real car is a single self-contained unit, not scattered parts.

Q2 **Class vs Object** Intermediate

What is the difference between a Class and an Object?

ANSWER

A Class is a blueprint or template that defines the properties (attributes/variables) and behaviors (methods/functions) that its objects will have — it doesn't occupy memory for actual data by itself.

An Object is a real, concrete instance of a class, created in memory, with actual values assigned to the properties defined in the class. You can create multiple objects from a single class, each with its own independent data.

EASY EXPLANATION: A Class is like the architectural blueprint for a house — it defines rooms, doors, and layout, but you can't live in a blueprint. An Object is an actual house built from that blueprint — you can build many houses (objects) from the same blueprint (class), and each house can be painted a different color or furnished differently (have different attribute values).

Q3 **Encapsulation** Intermediate

What is Encapsulation, and why is it useful?

ANSWER

Encapsulation is the practice of bundling data (variables) and the methods that operate on that data into a single unit (a class), while restricting direct access to some of the object's internal details — typically by making variables private and exposing controlled access through public getter/setter methods.

It's useful because it protects data from unintended or invalid modification, hides implementation details so internal logic can change without breaking external code, and makes code easier to maintain and debug.

EASY EXPLANATION: Think of a capsule medicine — the actual medicine (data) is sealed inside a protective shell (class), and you can't directly touch or tamper with the raw medicine. You interact with it only through the intended way (swallowing the capsule), which protects both the medicine and you.

Q4 **Abstraction** Intermediate

What is Abstraction, and how is it different from Encapsulation?

ANSWER

Abstraction is the concept of showing only essential, relevant features of an object to the outside world while hiding unnecessary implementation complexity — focused on WHAT an object does, not HOW it does it.

The key difference: Abstraction is about DESIGN — hiding complexity by exposing only necessary details (achieved via abstract classes/interfaces). Encapsulation is about IMPLEMENTATION — hiding data by restricting direct access (achieved via access modifiers like private/public). Abstraction hides complexity; Encapsulation hides data.

EASY EXPLANATION: When you drive a car, you use the steering wheel, accelerator, and brake without needing to know how the engine, fuel injection, or transmission actually works internally — that's Abstraction (hiding the complex 'how'). Encapsulation is the fact that the engine parts are sealed inside the hood, so you can't randomly poke at wires while driving — it protects the internal data/mechanism.

Q5 **Inheritance** Intermediate

What is Inheritance, and what are its types?

ANSWER

Inheritance is a mechanism where a new class (child/derived/subclass) acquires the properties and behaviors (fields and methods) of an existing class (parent/base/superclass), promoting code reuse and establishing an 'is-a' relationship between classes.

Common types: Single Inheritance (one child inherits from one parent), Multilevel Inheritance (a chain: grandparent -> parent -> child), Hierarchical Inheritance (multiple children inherit from one parent), and Multiple Inheritance (a child inherits from more than one parent — supported directly in some languages like C++, but achieved via interfaces in languages like Java, which don't allow multiple class inheritance directly to avoid ambiguity).

EASY EXPLANATION: Think of a family: a child inherits certain traits from their parents (like eye color or last name) without having to 're-learn' or redefine them from scratch. In code, a 'Dog' class can inherit basic traits like 'eat()' and 'sleep()' from an 'Animal' class, then just add its own extra behavior like 'bark()'.

Q6 **Polymorphism** Intermediate

What is Polymorphism, and what are its types?

ANSWER

Polymorphism means 'many forms' — it allows objects of different classes to be treated through a common interface, where the same method name/call can behave differently depending on the actual object it's invoked on.

Compile-Time (Static) Polymorphism - achieved through Method Overloading: multiple methods with the SAME name but DIFFERENT parameters (number/type) within the same class; resolved by the compiler at compile time.

Run-Time (Dynamic) Polymorphism - achieved through Method Overriding: a child class provides its own specific implementation of a method already defined in its parent class; resolved at runtime based on the actual object type.

EASY EXPLANATION: Imagine pressing the 'accept' button in different apps — in a food delivery app it confirms an order, in a call app it picks up the phone, in a game it accepts a friend request. Same button/action name ('accept'), but the actual behavior depends entirely on which app (object) you're using — that's polymorphism.

Q7 **Method Overloading vs Overriding** Intermediate

What is the difference between Method Overloading and Method Overriding?

ANSWER

Method Overloading happens WITHIN the same class — multiple methods share the same name but differ in parameters (number, type, or order). It's resolved at compile time and doesn't involve inheritance.

Method Overriding happens BETWEEN a parent and child class — the child class redefines a method that already exists in the parent class, with the EXACT same name and parameters. It's resolved at runtime and requires inheritance, allowing the child's specific version to run instead of the parent's.

EASY EXPLANATION: Overloading is like having several doorbells at the same house that all ring different tones depending on which button (parameters) you press — but they're all still 'the doorbell' at one house (one class). Overriding is like a franchise restaurant chain where each individual branch (child class) can customize the 'signature dish' recipe (method) while keeping the same dish name as the original headquarters (parent class) recipe.

Q8 **Constructor** Intermediate

What is a Constructor, and what are its key characteristics?

ANSWER

A constructor is a special method automatically called when an object is created, primarily used to initialize the object's attributes/state with initial values.

Key characteristics: it has the SAME name as the class, it has NO return type (not even void), it's called automatically at object creation, and it can be overloaded (multiple constructors with different parameters, known as constructor overloading). If no constructor is defined, most languages automatically provide a 'default constructor'.

EASY EXPLANATION: A constructor is like the initial setup checklist when a new employee joins a company — before they can actually start working (before the object is usable), certain basic setup

steps (assigning a desk, an email ID, initial values) must run automatically the moment they're 'created' as an employee.

Q9 **Access Modifiers** Intermediate

What are Access Modifiers (public, private, protected)?

ANSWER

Access modifiers control the visibility/accessibility of classes, methods, and variables from other parts of a program, enforcing encapsulation.

Public - accessible from anywhere, both inside and outside the class.

Private - accessible only within the same class; not visible to subclasses or outside code.

Protected - accessible within the same class AND by subclasses (even in different packages/files in some languages), but not by unrelated outside code.

Default/Package-Private (in languages like Java, when no modifier is specified) - accessible only within the same package.

EASY EXPLANATION: Think of a house: Public is like the front porch — anyone walking by can see and access it. Private is like your personal bedroom — only you (the class itself) can go in. Protected is like a shared family living room — family members (subclasses) can access it, but random outside visitors cannot.

Q10 **Abstract Class vs Interface** Intermediate

What is the difference between an Abstract Class and an Interface?

ANSWER

An Abstract Class can have BOTH abstract methods (no implementation, must be overridden) and concrete methods (with actual implementation), can have constructors, instance variables, and access modifiers, and a class can typically extend (inherit from) only ONE abstract class.

An Interface traditionally only declares method signatures (no implementation) that implementing classes MUST provide — though modern languages (like Java 8+) now also allow default/static methods with bodies. A class can implement MULTIPLE interfaces, making interfaces the way to achieve something like multiple inheritance in languages that don't allow it directly for classes.

EASY EXPLANATION: An Abstract Class is like a partially filled-out form template — some fields are already filled in for you (concrete methods), while others are left blank for you to fill (abstract methods), and you can only use one such template at a time. An Interface is more like a strict checklist of promises/requirements ('you MUST be able to do X, Y, Z') that a class agrees to fulfill — and a class can agree to multiple separate checklists (interfaces) at once.

Q11 **this vs super** Intermediate

What is the purpose of the 'this' and 'super' keywords?

ANSWER

'this' refers to the CURRENT object/instance of the class it's used in — commonly used to differentiate between instance variables and constructor/method parameters that share the same name, or to call another constructor within the same class (constructor chaining).

'super' refers to the immediate PARENT class of the current object — commonly used to call the parent class's constructor, or to access a parent class's method/variable that has been overridden or hidden by the child class.

EASY EXPLANATION: 'this' is like saying 'MY OWN [thing]' to be clear you mean your own version, not someone else's with the same name. 'super' is like saying 'my PARENT'S [thing]' — specifically reaching back up to what your parent class originally had, even if you've since overridden it with your own version in the child class.

Q12 **Association, Aggregation, Composition** Intermediate

What is the difference between Association, Aggregation, and Composition?

ANSWER

Association is a general relationship where two independent classes are connected/interact with each other, but neither owns the other (e.g., a Teacher and a Student are associated, but each can exist independently of the other).

Aggregation is a specialized 'HAS-A' relationship representing ownership, but the contained object CAN exist independently of the owner (a weak/loose relationship) — e.g., a Department 'has' Professors, but if the Department is deleted, the Professors still exist and can move elsewhere.

Composition is a stronger 'HAS-A' relationship where the contained object CANNOT exist independently of the owner (a strong/tight relationship) — e.g., a House 'has' Rooms, but if the House is destroyed, those specific Rooms cease to exist too.

EASY EXPLANATION: Association is like two people who know each other (a Teacher and a Student) but live completely separate lives. Aggregation is like a Team having Players — if the Team disbands, the Players still exist and can join another team. Composition is like a Human having a Heart — the Heart cannot exist meaningfully on its own outside that specific Human; if the Human 'ends', so does that particular Heart's role.

Advanced Level Questions

Q13 **Runtime vs Compile-Time Polymorphism (Deep Dive)** Advanced

How does Method Overriding achieve runtime polymorphism internally (dynamic method dispatch)?

ANSWER

When a method is overridden, the actual method that executes is determined at RUNTIME based on the actual object type (not the reference/variable type) — this is called Dynamic Method Dispatch or late binding.

Internally, most OOP languages implement this using a Virtual Method Table (vtable) — each class with overridable methods has a table of pointers to the actual method implementations. When a method is called on an object, the program looks up the correct implementation in that object's vtable at runtime, rather than deciding which method to call ahead of time at compile time (as happens with overloading).

EASY EXPLANATION: Imagine you have a generic remote labeled 'TV Remote' (the reference type), but it's actually controlling a specific Samsung TV (the actual object type) at that moment. Even though the remote LOOKS generic, pressing 'power' triggers whatever the ACTUAL connected TV (object) does — if you later point the same generic remote at a Sony TV instead, the SAME button press now behaves according to the Sony TV's own behavior, decided at the moment of pressing (runtime), not printed permanently on the remote itself.

Q14 **Diamond Problem** Advanced

What is the Diamond Problem in Multiple Inheritance, and how is it resolved?

ANSWER

The Diamond Problem occurs when a class inherits from two classes that both inherit from a common base class, and the base class has a method that both intermediate classes override differently — creating ambiguity about WHICH version the final child class should inherit, since there are two conflicting paths up to the same ancestor (forming a diamond shape in the inheritance diagram).

Resolution: languages like Java avoid this entirely by NOT allowing multiple inheritance of classes (only single class inheritance, but multiple interface implementation, where default method conflicts must be explicitly resolved by the programmer). C++ resolves it using 'Virtual Inheritance', ensuring only ONE shared copy of the common base class exists, so there's no ambiguity about which version to use.

EASY EXPLANATION: Imagine a child has two parents, and BOTH parents separately taught them a slightly different version of the 'family recipe' (originally from a shared grandparent). When the child tries to cook that dish, which version do they use — Mom's twist or Dad's twist? That confusion is the Diamond Problem. Some cultures avoid this entirely by having only one recognized parental lineage teach recipes (Java's approach); others have a strict family rule that ensures everyone ultimately agrees to use just the ORIGINAL grandparent's one true recipe (C++'s virtual inheritance).

Q15 **SOLID Principles** Advanced

What are the SOLID principles of Object-Oriented Design?

ANSWER

S - Single Responsibility Principle: a class should have only ONE reason to change, i.e., it should do just one job/responsibility.

O - Open/Closed Principle: classes should be OPEN for extension but CLOSED for modification — you should be able to add new behavior without changing existing, already-tested code.

L - Liskov Substitution Principle: objects of a subclass should be substitutable for objects of the parent class without breaking the correctness of the program.

I - Interface Segregation Principle: clients shouldn't be forced to depend on methods/interfaces they don't actually use — prefer several small, specific interfaces over one large, general-purpose one.

D - Dependency Inversion Principle: high-level modules shouldn't depend directly on low-level modules — both should depend on abstractions (interfaces), not concrete implementations.

EASY EXPLANATION: Think of a well-run restaurant kitchen: Single Responsibility = each chef has one clear station (grill, salad, dessert), not everything at once. Open/Closed = you can add a new dish to the menu without rewriting the entire existing menu. Liskov Substitution = any 'chef' assigned to a station should be able to do that job correctly, no surprises. Interface Segregation = a dessert chef isn't forced to also know grill equipment they'll never use. Dependency Inversion = the kitchen depends on the general concept of 'a qualified chef', not one specific named person, so staff can be swapped without breaking the kitchen.

Q16 **Virtual Functions & Pure Virtual Functions (C++ context)** Advanced

What is a Virtual Function, and what is a Pure Virtual Function?

ANSWER

A Virtual Function is a member function declared in a base class that you EXPECT to be overridden in derived classes, enabling runtime polymorphism — when called through a base class pointer/reference, it executes the derived class's overridden version instead of the base class's own version.

A Pure Virtual Function has NO implementation in the base class itself (declared with '= 0' in C++) — it forces every derived class to provide its own implementation. A class containing at least one pure virtual function becomes an 'abstract class' and cannot be instantiated directly on its own.

EASY EXPLANATION: A Virtual Function is like a job description that says 'greet the customer' with a DEFAULT suggested greeting provided, but each store branch is free to customize it if they want. A Pure Virtual Function is a job description that says 'greet the customer' with NO suggested script at all — it's mandatory that every branch MUST write their own greeting; there's no usable 'headquarters default' to fall back on, and you can't open a nameless prototype store based only on the incomplete job description itself.

Q17 **Object Slicing (C++ context)** Advanced

What is Object Slicing?

ANSWER

Object Slicing occurs in C++ when a derived class object is assigned to a base class object (by value, not by pointer/reference) — only the base class portion of the data gets copied, and the extra derived-class-specific data/behavior is 'sliced off' and lost.

This typically happens accidentally when passing derived objects to functions that expect a base class parameter BY VALUE, or storing derived objects directly in a container of base class type — the fix is to use pointers or references to the base class instead, which preserve the full derived object.

EASY EXPLANATION: Imagine photocopying a colorful 3D pop-up greeting card using a plain black-and-white flatbed scanner — the scan (base class copy) only captures the flat, generic outline, and all the extra special pop-up features/color (derived class specifics) get physically flattened and lost in that particular copy, even though the original card still exists fully intact elsewhere.

Q18 **Static Binding vs Dynamic Binding** Advanced

What is the difference between Static Binding and Dynamic Binding?

ANSWER

Static Binding (early binding) means the compiler decides WHICH method/function to call at COMPILE TIME, based on the reference/variable type — used for overloaded methods, static methods, and private methods, which cannot be overridden.

Dynamic Binding (late binding) means the decision of which method to call is deferred until RUNTIME, based on the ACTUAL object type — used for overridden (virtual) methods, enabling runtime polymorphism.

EASY EXPLANATION: Static Binding is like pre-printing your entire travel itinerary before the trip even starts, based only on your booking type — it's locked in and decided ahead of time (compile time). Dynamic Binding is like deciding your actual route in real-time based on live traffic conditions once you're actually driving (runtime) — the final decision depends on the real, current situation, not something fixed in advance.

Q19 **Coupling and Cohesion** Advanced

What is Coupling and Cohesion in OOP design, and why does low coupling with high cohesion matter?

ANSWER

Coupling refers to the degree of interdependence between different classes/modules — LOW coupling means classes are relatively independent and changes in one rarely require changes in another; HIGH coupling means classes are tightly interlinked, so a change in one often forces changes throughout others.

Cohesion refers to how closely related and focused the responsibilities WITHIN a single class/module are — HIGH cohesion means a class does one well-defined job with all its methods/data closely related to that single purpose; LOW cohesion means a class handles many unrelated responsibilities crammed together.

Good design aims for LOW coupling (classes don't unnecessarily depend on each other's internals) and HIGH cohesion (each class has one clear, focused job) — this makes code easier to test, reuse,

maintain, and modify without unexpected ripple effects.

EASY EXPLANATION: Low Coupling is like having separate, independent departments in a company (Sales, HR, IT) where each can reorganize its own internal process without forcing every other department to change too. High Cohesion is like each department only handling tasks that actually belong to it — HR doesn't also handle random IT support tickets and marketing campaigns all at once; it stays tightly focused on HR-related work only.

Q20	**Design Patterns (Singleton Example)**	Advanced

What is a Design Pattern? Explain the Singleton Pattern as an example.

ANSWER

A Design Pattern is a general, reusable, proven solution template to a commonly occurring software design problem — it's not finished code you copy-paste directly, but a conceptual blueprint/approach you adapt to your specific situation.

The Singleton Pattern ensures a class has ONLY ONE instance throughout the entire application's lifetime, and provides a single, well-known global point of access to that one instance. It's implemented by making the constructor private (so outside code can't create new instances directly), keeping a single static instance variable inside the class itself, and exposing a public static method that returns that same existing instance every time it's called (creating it only once, the very first time it's needed).

EASY EXPLANATION: A Design Pattern is like a proven, standard recipe technique (e.g., 'how to properly caramelize onions') that experienced chefs reuse across many different dishes, rather than something copy-pasted from one exact dish. The Singleton Pattern specifically is like a company having exactly ONE official CEO at any given time — no matter how many different departments ask 'who's the CEO', they're always directed to and receive the SAME single person, and nobody outside is allowed to just walk in and personally appoint a brand new additional CEO themselves.